Progetto 1: Server HTTP Concorrente

Gestione Prenotazione Aule Studio in C

Massimo Mantineo (541924) Pierluca Tino Castorina (545101)

Università degli Studi di Messina
Dipartimento MIFT - Laurea in Informatica

Anno Accademico 2024/2025

1. Obiettivi del Progetto

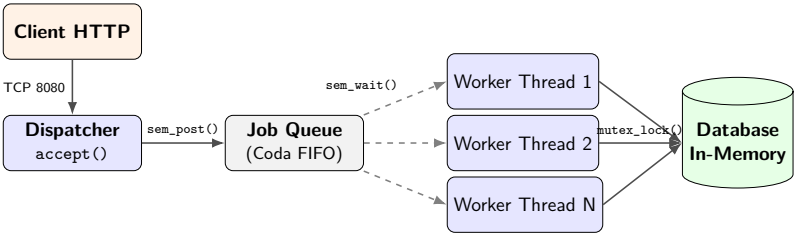
Il Problema

Implementare un server per la gestione di prenotazioni in grado di servire simultaneamente molteplici client, garantendo la coerenza dei dati e riducendo l'overhead computazionale di sistema.

Le Soluzioni Adottate:

- **C Nativo & POSIX:** Utilizzo delle *system call* di basso livello anziché di framework moderni (Node.js, Spring) per massimizzare il controllo delle risorse.
- **Architettura a Thread Pool:** Sostituzione del pattern classico basato su `fork()` con un pool di worker thread pre-allocati.
- **Sincronizzazione Esplicita:** Impiego combinato di Semafori (per la comunicazione di rete) e Mutex (per l'accesso in mutua esclusione al database in memoria).

2. Architettura a Thread Pool



Evidenza: Handshake TCP

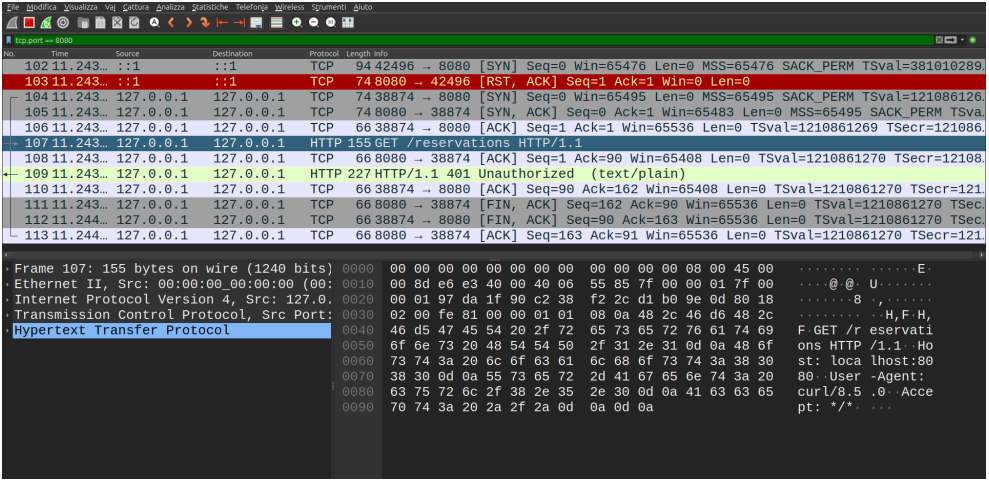


Figura: Tracciato Wireshark del Three-Way Handshake TCP.

3. Il Pattern Produttore-Consumatore

Per interfacciare in modo sicuro la rete (Main Thread) e l'esecuzione (Worker Threads), si sfrutta una **Job Queue** regolata da Semafori, prevenendo i colli di bottiglia e il *busy-waiting*.

Dispatcher (Produttore)

```
// Accettazione connessione client
client_sock = accept(server_fd, ...);
task->socket = client_sock;

// Aggiunta in coda critica
pthread_mutex_lock(&pool->queue_mutex);
pool->queue_tail->next = job;
pthread_mutex_unlock(&pool->queue_mutex);

// Incremento del semaforo
sem_post(&pool->queue_sem);
```

Worker (Consumatore)

```
while (1) {
    // Il thread e' in sleep se la coda e'
        vuota
    sem_wait(&pool->queue_sem);

    // Estrazione in mutua esclusione
    pthread_mutex_lock(&pool->queue_mutex);
    job_t* job = pool->queue_head;
    // ...
    pthread_mutex_unlock(&pool->queue_mutex);

    // Parsing ed esecuzione
    handle_client(job);
}
```

4. Thread Safety sulle Prenotazioni

Tutte le chiamate API che leggono o modificano la lista delle aule sono protette mediante un costrutto **Mutex** centralizzato. Questo garantisce l'isolamento transazionale ed evita corruzioni (Data Race).

```
int rm_create(resource_manager_t* rm, const char* room, const char* student) {
    pthread_mutex_lock(&rm->mutex); // ACQUISIZIONE LOCK

    if (rm->count >= MAX_RESERVATIONS) {
        pthread_mutex_unlock(&rm->mutex);
        return -1;
    }

    // Inserimento sicuro dei dati nella memoria condivisa
    strncpy(rm->reservations[rm->count].room, room, 127);
    strncpy(rm->reservations[rm->count].student, student, 127);
    rm->count++;

    pthread_mutex_unlock(&rm->mutex); // RILASCIO LOCK
    return rm->count - 1;
}
```

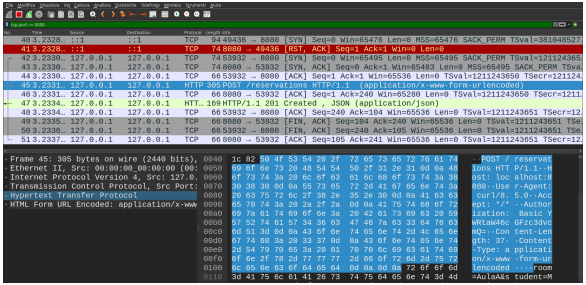
5. Specifiche API REST (CRUD)

Metodo	Endpoint	Azione Interna	Status Code
`GET`	`/reservations`	Serializzazione e lettura in sola lettura	`200 OK`
`POST`	`/reservations`	Validazione Body URL-Encoded, Aggiunta Struct	`201 Created`
`PUT`	`/reservations/{id}`	Ricerca indice array e sovrascrittura valori	`200 OK`
`DELETE`	`/reservations/{id}`	Rimozione elemento e traslazione a sinistra (shift)	`204 No Content`

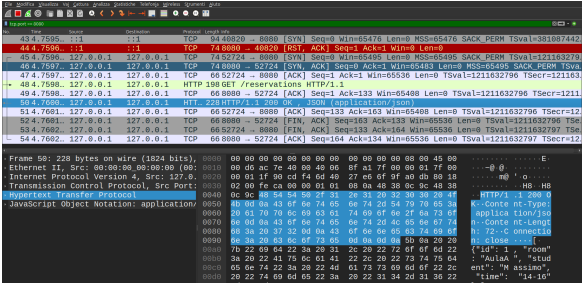
Autenticazione

Le rotte sensibili richiedono l'header `Authorization: Basic` (token Base64).

Evidenza: Autenticazione e JSON



POST Request e Basic Auth



Risposta 200 OK (JSON)

6. Ottimizzazioni Architetturali

Sincronizzazione Centralizzata (Mutex)

L'architettura previene attivamente *race conditions* e corruzione dei dati in memoria. L'impiego rigoroso del **Mutex** (`pthread_mutex_t`) garantisce che l'accesso alla memoria condivisa avvenga in totale mutua esclusione per ogni Worker Thread, assicurando l'assoluta coerenza transazionale sia in lettura che in scrittura.

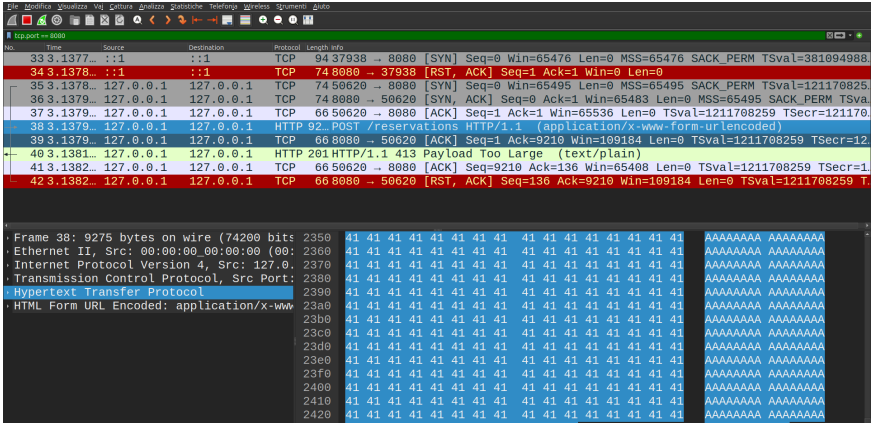
De-frammentazione della Memoria Dinamica

Poiché il C è sprovvisto di Garbage Collector, la rimozione logica di una prenotazione (`DELETE`) espone al rischio di frammentazione del vettore. L'implementazione impiega uno **shift a sinistra** progressivo tramite un **ciclo iterativo for** all'interno dell'area protetta, garantendo la ricompattazione continua dell'array dinamico senza librerie esterne.

7. Sicurezza e Gestione Vulnerabilità

Mitigazione delle criticità intrinseche in ambiente C/POSIX:

- **Socket Leaks:** Dismissione sicura `close(fd)`.
- **Buffer Overflows:** Varianti sicure, errore 413.
- **Interruzione SIGPIPE:** Flag `MSG_NOSIGNAL`.



8. Analisi e Load Testing

Il progetto è stato collaudato sfruttando lo strumento standard **Apache Bench (ab)**. La simulazione ha previsto: **10.000 richieste totali** con un livello di concorrenza di **100 thread paralleli**.

Metriche di Prestazione

- **Total Requests:** 10.000
- **Concorrenza Esterna:** 100
- **Success Rate:** 100% (0 Errori)
- **Throughput:** $\sim$ 33.229 Req/Sec

Interpretazione Architetturale

L'assenza totale di connessioni droppate dimostra che la coda semaforica regge il carico e previene i fallimenti TCP. Inoltre, il tempo di risposta mediano di appena **3.0 ms** per richiesta conferma la leggerezza dei thread worker e la corretta implementazione dei lock Mutex sulle risorse condivise.

9. Conclusioni e Sviluppi Futuri

L'implementazione in linguaggio C di un Thread Pool associato a socket BSD si è dimostrata una soluzione *memory-safe* ad altissime prestazioni per la costruzione di architetture HTTP concorrenti.

Sviluppi per Ambienti di Produzione:

1. **Persistenza dei Dati**: Passaggio dalla struttura in RAM ad un database relazionale leggero (come `SQLite3`) per mantenere lo stato dopo il riavvio del processo.
2. **I/O Asincrono (`epoll`)**: Sostituzione delle chiamate bloccanti `read()` con un loop *Event-Driven* nativo Linux per supportare il limite dei 10.000 Socket attivi.
3. **Cifratura**: Implementazione del protocollo *TLS/SSL* tramite librerie come *OpenSSL* per esporre API sicure (HTTPS).

Grazie per l'attenzione.